Zero Trust Network Access (ZTNA) with Netskope Private Access — VPN Replacement

Implementation Solution Guide

Version 1.0.0 · 2026-06-29

Contents

Executive Summary — Zero Trust Network Access (ZTNA) with Netskope Private Access 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 4

Outcome . . . 4

1. Solution Overview — Zero Trust Network Access (ZTNA) with Netskope Private Access 5

What ZTNA / Netskope Private Access is . . . 5

Why this matters (the *why* lives here) . . . 5

How it works . . . 5

What you can control . . . 5

How this differs from a VPN . . . 6

Limitations (be honest with buyers) . . . 6

The standard deployment model . . . 6

2. Requirements Guide — Zero Trust Network Access (ZTNA) with Netskope Private Access 6

2.1 Netskope Private Access entitlement . . . 6

2.2 Publisher (connector) host capacity . . . 7

2.3 Private application inventory . . . 7

2.4 Entra ID groups for app entitlement . . . 7

2.5 The Netskope client deployed to endpoints . . . 7

2.6 Device posture signals . . . 7

2.7 DNS resolution for private apps . . . 7

2.8 A pilot group and a VPN coexistence window . . . 8

Naming convention (use verbatim everywhere) . . . 8

Checklist . . . 8

3. Implementation Guide — Zero Trust Network Access (ZTNA) with Netskope Private Access 9

Step 1 — Assess: confirm prerequisites and inventory . . . 9

Step 2 — Configure: build the entitlement groups . . . 9

Step 3 — Configure: deploy the publisher (connector) . . . 9

Step 4 — Configure: publish the private apps . . . 10

Step 5 — Configure: author least-privilege access policy . . . 10

Step 6 — Pilot: deploy the client and steer the pilot users . . . 10

Step 7 — Validate: prove the pilot against the gate . . . 10

Step 8 — Expand: widen entitlement and add production publishers . . . 11

Step 9 — Cut over: retire the VPN . . . 11

5. Testing Guide — Zero Trust Network Access (ZTNA) with Netskope Private Access 11

Test 1 — Publisher healthy . . . 11
Test 2 — Client steering on the endpoint . . . 11
Test 3 — Entitled app is reachable . . . 12
Test 4 — Least privilege enforced (non-entitled app denied) . . . 12
Test 5 — Posture gate enforced . . . 12
Test 6 — Identity/group binding correct . . . 12
Test 7 — VPN coexistence and fall-back . . . 12
Test 8 — Access and usage report . . . 12
Pass/fail gate . . . 12
6. Troubleshooting Guide — Zero Trust Network Access (ZTNA) with Netskope Private Access
A. A published app is unreachable . . . 13
B. User reaches the wrong set of apps . . . 13
C. Posture check blocks a healthy device . . . 13
D. Client steering problems . . . 13
E. Coexistence and cutover issues . . . 14
F. Diagnostics quick reference . . . 14
G. When to escalate . . . 14
7. Best Practices — Zero Trust Network Access (ZTNA) with Netskope Private Access 14
8. Deliverables Summary — Zero Trust Network Access (ZTNA) with Netskope Private Access
Documentation . . . 15
PowerShell scripts . . . 15
What the buyer receives . . . 16
Outcome . . . 16
Appendix A — PowerShell Scripts 17

Executive Summary — Zero Trust Network Access (ZTNA) with Netskope Private Access

Network · VPN Replacement

The legacy VPN hands a user the whole network — a slow, hard-to-scale model and a favorite attacker on-ramp, where one stolen credential or one compromised laptop can pivot across everything the route reaches. Zero Trust Network Access with Netskope Private Access replaces it: deploy a `publisher` next to your private apps, publish each app individually, and grant identity- and posture-based least-privilege access — users reach the specific apps they are entitled to, and the network is never exposed.

This is a complete, deploy-it-the-repeatable-way package for implementing ZTNA with Netskope Private Access and migrating off the VPN, written for an administrator with no prior ZTNA experience. It is authored at the correct architectural and procedural level; Netskope console specifics are deferred to the official Netskope documentation.

What you get

• A plain-language Solution Overview of ZTNA, `publishers`, private-app publishing, identity- and posture-based access, and how it differs from a VPN.

• A Requirements Guide with verify / implement-if-missing for every prerequisite, plus a naming convention applied to every artifact.

• An Implementation Guide of precise, ordered steps: deploy `publishers`, publish private apps, author least-privilege access policy, pilot, validate, expand, and cut over from the VPN.

• Five PowerShell scripts for ZTNA-readiness discovery, access validation, an access and usage report, safe remediation of broken access, and an entitlement-group builder.

• Testing and Troubleshooting guides for a confident, access-safe rollout.

What this delivers

Least-privilege, per-app access — each private app published and reachable only by entitled identities on healthy, posture-checked devices, with no inbound network exposure — and a staged VPN coexistence-then-cutover migration that proves every app before the VPN is retired.

The deployment model

Assess to Configure to Pilot to Validate to Expand — with a pilot gate and an access-affecting rollback (fall back to the VPN) at every stage.

Outcome

Users reaching only the private apps they are entitled to, identity- and posture-based, with the network hidden — and the legacy VPN retired, proven on a pilot before you scale.

1. Solution Overview — Zero Trust Network Access (ZTNA) with Netskope Private Access

What ZTNA / Netskope Private Access is

Zero Trust Network Access (ZTNA) replaces the legacy remote-access VPN. Instead of placing a user *on the network* and trusting everything they can reach, ZTNA brokers access to one private application at a time, after checking *who* the user is and *what state* their device is in. Netskope Private Access is the ZTNA service that implements this: lightweight `publishers` (connectors) are deployed next to your private apps, each private app is published as a named object, and an access policy decides — per identity, per group, per device posture — exactly which apps each user may reach. The network itself is never exposed.

Why this matters (the *why* lives here)

A VPN hands the user a route into the corporate network. Once connected, a compromised laptop or a stolen credential can scan and pivot across everything that route reaches — the classic attacker on-ramp. VPN concentrators are also a scaling and performance bottleneck (all traffic hairpins through them) and they are hard to govern (access is by subnet, not by app or identity). ZTNA inverts the model: there is no inbound route to expose, apps are invisible until policy grants them, and access is least-privilege by default — a user reaches only the specific apps their identity and device posture entitle them to, and nothing else. That is the reason to make the switch; the rest of this package is the repeatable way to do it safely.

How it works

1. A `publisher` (connector) is deployed inside each network segment that hosts private apps. It makes an *outbound* connection to the Netskope cloud — no inbound firewall ports are opened.
1. Each private app (an internal web app, an RDP/SSH host, a file share, a thick-client service) is published as a named private-application object that maps to its internal host(s) and port(s).
1. The Netskope client on the endpoint steers traffic for published apps to the nearest Netskope point of presence, which brokers the session to the right `publisher`.
1. An access policy evaluates the user's identity and group (from Entra ID) and the device posture (managed, compliant, certificate present, etc.) and grants or denies the specific app.
1. Because brokering is per-app and identity/posture-gated, the user reaches only entitled apps — the network is never routable end to end.
1. The VPN and ZTNA run side by side during a coexistence window, then the VPN is cut over and retired once the published app set is proven.

What you can control

- Which private apps are published, and the exact internal hosts/ports each one maps to.

- Which identities and Entra ID groups may reach each app (least-privilege, per-app access).
- Device posture conditions that gate access (managed/compliant device, certificate, OS state).
- Where `publishers` are placed (per site / per segment) for performance and resilience.
- The coexistence-then-cutover schedule that retires the VPN once apps are proven.

How this differs from a VPN

- VPN grants network reachability (a route to a subnet); ZTNA grants application access (one app).
- VPN trusts the connection; ZTNA re-checks identity and posture for every app, every time.
- VPN exposes a public concentrator; ZTNA `publishers` dial out, so nothing is exposed inbound.
- VPN access is hard to audit by app; ZTNA access maps cleanly to named apps and named groups.

Limitations (be honest with buyers)

- Some legacy or non-TCP applications need specific handling; inventory and test each app before cutover rather than assuming parity with the VPN.
- ZTNA brokers private-app access; it is not a full secure web gateway by itself (web/SaaS traffic is a separate Netskope capability outside this solution's scope).
- The exact Netskope console screens and menu paths change over time and are deferred to the official Netskope documentation and your tenant — this guide names the step and the artifact, and you confirm the precise control in your console.

The standard deployment model

> *Assess the app inventory and prerequisites, configure* `publishers` *and private apps and policy, pilot with a small group, validate access and posture, then expand and cut over from the VPN. Deployment model: Assess to Configure to Pilot to Validate to Expand.*

2. Requirements Guide — Zero Trust Network Access (ZTNA) with Netskope Private Access

Verify every requirement before configuring. Each entry: Description, Why, Verify, and Implement-if-missing. Category-wide prerequisites are in `global-requirements-network.md`; the items below are specific to ZTNA / Netskope Private Access.

2.1 Netskope Private Access entitlement

Description: The Netskope tenant has the Private Access (ZTNA) module enabled and licensed. Why: Private Access is the service that publishes apps and brokers per-app access; without it there is nothing to configure. Verify: Sign in to your Netskope tenant admin console and confirm the Private Access capability is present and licensed (defer the exact location to the Netskope documentation). Implement: Engage your Netskope account team or reseller to enable and license the Private Access module on the tenant.

2.2 Publisher (connector) host capacity

Description: A supported host (VM or appliance) in each network segment that hosts private apps, on which a `publisher` can run and make outbound connections to the Netskope cloud. Why: The `publisher` is the data-plane connector that reaches your internal apps; it must sit where it can resolve and route to those apps and dial outbound to Netskope. Verify: Confirm a candidate host exists per site/segment with outbound internet egress; run `NSZTNA-Discovery.ps1` to record candidate hosts and their reachability. Implement: Provision a host meeting Netskope's `publisher` sizing, allow outbound egress to the Netskope cloud, then deploy the `publisher` (deferred to Netskope documentation for the exact image and steps).

2.3 Private application inventory

Description: A documented inventory of every private app to be reached without the VPN — name, internal host(s)/FQDN, port(s), and protocol. Why: Each app becomes a named private-application object mapped to specific hosts/ports; an incomplete inventory is the main cause of broken access at cutover. Verify: Review the inventory; run `NSZTNA-Discovery.ps1` to capture VPN routes/hosts currently in use as a starting list. Implement: Build the inventory from VPN split-tunnel routes, DNS, and app owners; record each app as `Private App – <App> – Inventory`.

2.4 Entra ID groups for app entitlement

Description: Microsoft Entra security groups that map users to the apps they are entitled to. Why: Access policy is identity- and group-based; clean groups make least-privilege access repeatable and auditable. Verify: Entra portal, Groups; run `NSZTNA-Validation.ps1` to confirm the entitlement groups exist and have members. Implement: Create the groups (see `NSZTNA-Set-AccessGroupTargeting.ps1`), naming each `Access Group – <App or Role> – <Stage>`.

2.5 The Netskope client deployed to endpoints

Description: The Netskope client (steering agent) installed and enrolled on target endpoints. Why: The client steers private-app traffic to Netskope so sessions can be brokered per app; without it the endpoint has no path to published apps. Verify: On a pilot device, confirm the client is installed and enrolled; run `NSZTNA-Validation.ps1` to check client presence/state. Implement: Deploy the client via Intune (see global requirement G3) and enroll the device.

2.6 Device posture signals

Description: A device-posture source (managed/compliant state, certificate, or Netskope posture check) usable in the access policy. Why: Least-privilege access is identity- and posture-based; posture ensures only healthy, managed devices reach sensitive apps. Verify: Confirm devices are Intune-enrolled and compliant (`https://intune.microsoft.com`); run `NSZTNA-Discovery.ps1` to record managed/compliance posture. Implement: Enforce Intune compliance policies and deploy any required certificate, then reference the posture in the access policy.

2.7 DNS resolution for private apps

Description: Endpoints (via Netskope) can resolve the internal names of published apps. Why: Private apps are reached by name; name resolution must work through the ZTNA path, not only on the old VPN. Verify: From a pilot device with the client active, resolve a published app's FQDN; run `NSZTNA-Validation.ps1` to test resolution/reachability of a published app. Implement: Configure private DNS for the published apps per Netskope guidance (deferred to Netskope documentation).

2.8 A pilot group and a VPN coexistence window

Description: An Entra security group of pilot users/devices, and an agreed window during which the VPN and ZTNA run side by side before cutover. Why: Access changes are disruptive if rushed; coexistence lets the pilot prove every app before the VPN is retired, and provides the rollback path (fall back to VPN). Verify: Entra, Groups: the pilot group `Access Group – Pilot – Pilot` exists; the coexistence window and rollback are approved (global requirement G7). Implement: Create the pilot group (`NSZTNA-Set-AccessGroupTargeting.ps1`) and document the coexistence schedule and the fall-back-to-VPN rollback.

Naming convention (use verbatim everywhere)

Every artifact is named `<Purpose> – <Scope> – <Stage>` and reused exactly across all guides:

- `Publisher – <Site> – Pilot` / `Publisher – <Site> – Production` — `publisher` (connector) per site.
- `Private App – <App> – Pilot` / `Private App – <App> – Production` — published private-application object.
- `Access Policy – <App or Role> – Pilot` / `Access Policy – <App or Role> – Production` — per-app access rule.
- `Access Group – <App or Role> – Pilot` / `Access Group – <App or Role> – Production` — Entra entitlement group.

Checklist

- Netskope Private Access module enabled and licensed
- `Publisher` host capacity per site/segment with outbound egress
- Private application inventory documented (host/port/protocol)
- Entra entitlement groups created — `NSZTNA-Validation.ps1`
- Netskope client deployed and enrolled on endpoints
- Device posture (Intune compliance / certificate) available to policy
- DNS resolves published app names through the ZTNA path
- Pilot group + approved VPN coexistence window and rollback

3. Implementation Guide — Zero Trust Network Access (ZTNA) with Netskope Private Access

Follow these steps in order. Identity and group work is done in the Microsoft Entra admin center (`https://entra.microsoft.com`); endpoint work in the Microsoft Intune admin center (`https://intune.microsoft.com`). All Netskope Private Access work (deploying `publishers`, publishing private apps, and authoring access policy) is done in the Netskope tenant admin console — the exact screens and menu paths are deferred to the official Netskope documentation, so this guide names each step and the artifact to create and you confirm the precise control in your console.

> *Model: deploy a `publisher` next to the apps, publish each private app, grant identity- and posture-based least-privilege access, prove it on a pilot while the VPN still runs, then cut over. Reuse the artifact names from Requirements section 2 (Naming convention) verbatim. Pilot gate (do not pass without it): every pilot user reaches every entitled app through ZTNA, posture is enforced, and the VPN is no longer needed by the pilot — only then expand and cut over. Rollback note: this is access-affecting. At every stage the VPN remains the fall-back. If ZTNA access fails, leave or re-enable the user's VPN access and remove them from the ZTNA `Access Group` until the issue is resolved; do not retire the VPN until the cutover stage passes.*

Step 1 — Assess: confirm prerequisites and inventory

1. Run `NSZTNA-Discovery.ps1` to capture endpoint posture, current VPN routes/hosts, and candidate

`publisher` hosts. Reconcile its output against the private application inventory from Requirements 2.3.

1. Confirm every requirement in section 2 is met (Netskope Private Access licensed, `publisher` host

capacity, client deployable, posture available, DNS path planned).

Step 2 — Configure: build the entitlement groups

1. Create the pilot entitlement group `Access Group – Pilot – Pilot` and add a few representative

users/devices (use `NSZTNA-Set-AccessGroupTargeting.ps1` with `-WhatIf` first, then commit).

1. Create a per-app or per-role entitlement group for each app set, named

`Access Group – <App or Role> – Pilot` (and the matching `Access Group – <App or Role> – Production` for later expansion).

1. Run `NSZTNA-Validation.ps1` to confirm each group exists and has members.

Step 3 — Configure: deploy the publisher (connector)

1. In the Netskope console, deploy a `publisher` on the host prepared in Requirements 2.2, named

`Publisher – <Site> – Pilot`. It connects outbound to the Netskope cloud — no inbound ports are opened (deferred to Netskope documentation for the image and registration steps).

1. Confirm the `publisher` shows healthy/connected in the Netskope console before continuing.

2. For resilience, plan a second `publisher` per site (`Publisher – <Site> – Production`) to add at

the Expand stage.

Step 4 — Configure: publish the private apps

1. For each app in the inventory, create a private-application object named

`Private App – <App> – Pilot`, mapping it to the exact internal host(s)/FQDN and port(s)/protocol recorded in Requirements 2.3, and associate it with `Publisher – <Site> – Pilot`.

1. Confirm DNS for each published app resolves through the ZTNA path (Requirements 2.7).

2. Keep the published app set small for the pilot (the apps the pilot users actually need).

Step 5 — Configure: author least-privilege access policy

1. For each app (or role), create an access policy named `Access Policy – <App or Role> – Pilot` that:

• matches the identity/group `Access Group – <App or Role> – Pilot` (or the pilot group), and

• requires the device posture condition from Requirements 2.6 (managed/compliant or certificate),

• and grants only `Private App – <App> – Pilot` — nothing broader.

1. Default-deny everything else: a user reaches only the apps an explicit policy grants.

2. Confirm policy order so the least-privilege rules evaluate as intended (deferred to Netskope documentation for the policy editor specifics).

Step 6 — Pilot: deploy the client and steer the pilot users

1. Deploy the Netskope client to the pilot devices via Intune (Requirements 2.5) and confirm it is enrolled and steering.

1. Leave the VPN in place for the pilot users — coexistence: ZTNA and VPN run side by side so a failed app reach falls back to the VPN.

1. Run `NSZTNA-Validation.ps1` on a pilot device to confirm client state, group membership, posture,

and that a published app resolves and is reachable.

Step 7 — Validate: prove the pilot against the gate

1. Run `NSZTNA-Validation.ps1` for each pilot user/app: confirm entitled apps are reachable and non-entitled apps are not.

1. Run `NSZTNA-Report.ps1` to produce the access and usage report (who is entitled to what, posture,

and which apps the pilot is actually using through ZTNA).

1. Complete the Testing Guide. The pilot gate passes only when every pilot user reaches every entitled app through ZTNA, posture is enforced, and the pilot no longer relies on the VPN.

Step 8 — Expand: widen entitlement and add production publishers

1. Add the second `publisher` per site (`Publisher – <Site> – Production`) for resilience and capacity.
1. Promote each app to `Private App – <App> – Production` and each rule to `Access Policy – <App or Role> – Production`, targeting the `Access Group – <App or Role> – Production` groups; add users in waves using `NSZTNA-Set-AccessGroupTargeting.ps1` (`-WhatIf` first).
1. Re-run `NSZTNA-Validation.ps1` and `NSZTNA-Report.ps1` after each wave; use `NSZTNA-Remediation.ps1` (default `-WhatIf`) to fix broken access (e.g. a user missing from an entitlement group).

Step 9 — Cut over: retire the VPN

1. Only after each wave passes the gate, remove that wave's users from VPN access (keep the VPN reachable as fall-back until the wave is confirmed stable).
1. When all waves are cut over and stable, decommission the VPN concentrator/access.
2. Keep `NSZTNA-Report.ps1` on a schedule to audit entitlement and usage going forward.

Order of operations

```
Assess (Discovery) -> entitlement groups -> deploy publisher -> publish apps
-> least-privilege policy -> deploy client + steer pilot (coexistence) ->
validate against gate -> expand in waves -> cut over and retire VPN
```

5. Testing Guide — Zero Trust Network Access (ZTNA) with Netskope Private Access

Run after the pilot is steered (Implementation Step 6) and again after each expansion wave and before each VPN cutover. Netskope console specifics are deferred to the official Netskope documentation; these tests confirm observable behavior.

Test 1 — Publisher healthy

Do: In the Netskope console, confirm `Publisher – <Site> – Pilot` shows connected/healthy.
Expected: The `publisher` is online and reachable by the Netskope cloud (no inbound ports opened).

Test 2 — Client steering on the endpoint

Do: On a pilot device run `NSZTNA-Validation.ps1` (also confirm the Netskope client is enrolled).
Expected: The client is installed, enrolled, and steering private-app traffic.

Test 3 — Entitled app is reachable

Do: As a pilot user entitled to an app, resolve and connect to `Private App – <App> – Pilot`; run `NSZTNA-Validation.ps1` to test resolution/reachability. Expected: The app resolves through the ZTNA path and the session connects.

Test 4 — Least privilege enforced (non-entitled app denied)

Do: As the same user, attempt to reach an app they are not entitled to. Expected: Access is denied — there is no broad network route; only explicitly granted apps work.

Test 5 — Posture gate enforced

Do: Attempt access from a non-compliant / unmanaged device (or simulate the posture failing). Expected: Access is denied or restricted per `Access Policy – <App or Role> – Pilot`; a healthy, compliant device succeeds.

Test 6 — Identity/group binding correct

Do: Run `NSZTNA-Validation.ps1` to confirm the user is in the expected `Access Group – <App or Role> – Pilot`, then confirm only that app set is reachable. Expected: Reachable apps match the user's entitlement groups exactly.

Test 7 — VPN coexistence and fall-back

Do: With the VPN still active, disable ZTNA access for a test user and confirm the VPN still serves the app; re-enable ZTNA. Expected: Coexistence works — the VPN is a valid fall-back until cutover.

Test 8 — Access and usage report

Do: Run `NSZTNA-Report.ps1`. Expected: A clear report of who is entitled to which apps, device posture, and which apps are being used through ZTNA.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1 to 6 pass for every pilot user/app; Test 7 confirms fall-back
Wave pass (before cutover)	Tests 1 to 6 and 8 pass for the wave's users and apps
Cutover ready	The wave passes and no app still depends on the VPN

6. Troubleshooting Guide — Zero Trust Network Access (ZTNA) with Netskope Private Access

Use `NSZTNA-Validation.ps1` for endpoint/identity state and `NSZTNA-Report.ps1` for entitlement and usage; use `NSZTNA-Remediation.ps1` (default `-WhatIf`) for safe fixes. Netskope console specifics are

deferred to the official Netskope documentation.

A. A published app is unreachable

Symptom	Cause	Remediation
Entitled app will not connect	`Publisher – <Site> – Pilot` down, or app object maps the wrong host/port	Confirm the `publisher` is healthy in the console; re-check the host/port/protocol on `Private App – <App> – Pilot`
App name will not resolve	DNS not configured through the ZTNA path	Fix private DNS for the app (Requirements 2.7); re-test with `NSZTNA-Validation.ps1`
Works on VPN, not on ZTNA	App not yet published, or user not in the entitlement group	Publish the app; add the user to `Access Group – <App or Role> – Pilot` with `NSZTNA-Remediation.ps1`

B. User reaches the wrong set of apps

• Too few apps: the user is missing from an `Access Group – <App or Role>` group — add them (run

`NSZTNA-Remediation.ps1 -WhatIf` first, then commit).

• Too many apps: a policy is too broad or default-deny is not in place — tighten

`Access Policy – <App or Role> – Pilot` to grant only its named `Private App` object.

C. Posture check blocks a healthy device

Symptom	Cause	Remediation
Compliant device denied	Device not reporting compliant/managed, or certificate missing	Confirm Intune compliance and any required certificate; re-evaluate posture
Access intermittently drops	Posture signal flapping	Stabilize the compliance/certificate signal; review the posture condition on the policy

D. Client steering problems

• Client not steering: confirm the Netskope client is installed and enrolled (`NSZTNA-Validation.ps1`);

redeploy via Intune if absent.

• Some traffic bypasses ZTNA: confirm the app is published and steered, and that no VPN split-tunnel

route still shadows it during coexistence.

E. Coexistence and cutover issues

• Users lose access at cutover: a wave was cut over before passing the gate — re-enable VPN access for

that wave, fix the failing app/policy, re-run the Testing Guide, then cut over again.

• VPN still in use after cutover: an app was never published — find it with `NSZTNA-Report.ps1` usage

data, publish it, validate, then remove the VPN dependency.

F. Diagnostics quick reference

```
.\NSZTNA-Discovery.ps1     # endpoint posture, VPN routes/hosts, candidate publisher hosts (read-only)
.\NSZTNA-Validation.ps1    # client state, group membership, posture, app resolution (read-only)
.\NSZTNA-Report.ps1        # entitlement + usage report (read-only)
.\NSZTNA-Remediation.ps1   # safe fixes, default -WhatIf (e.g. add user to entitlement group)
```

Netskope-side diagnostics (publisher health, session logs, policy hits) are in the Netskope console and are deferred to the official Netskope documentation.

G. When to escalate

If the `publisher` is healthy, the app object maps the correct host/port, DNS resolves through ZTNA, the user is in the right entitlement group with a compliant device, and an entitled app still will not connect, collect the Netskope session/policy logs for that user and app and open a Netskope support case. Keep the affected users on the VPN fall-back until resolved.

7. Best Practices — Zero Trust Network Access (ZTNA) with Netskope Private Access

• Publish the app, not the network. Map each private app to its exact host(s)/port(s) and grant that

app alone — never a subnet. This per-app model is the whole point of ZTNA.

• Default-deny, then grant. Every user should reach only the apps an explicit policy grants; nothing

is reachable by default. Audit for any over-broad rule that recreates VPN-style network access.

• Gate on identity and posture together. Bind each `Access Policy` to an Entra entitlement group and

a device-posture condition (managed/compliant or certificate) so only healthy, known devices reach sensitive apps.

• Name everything `<Purpose> – <Scope> – <Stage>` and reuse the names verbatim — `Publisher – <Site> – Pilot`,

`Private App – <App> – Pilot`, `Access Policy – <App or Role> – Pilot`, `Access Group – <App or Role> – Pilot`. Consistent names make access auditable and the rollout repeatable.

• Build the app inventory first. The number-one cause of broken cutovers is an app nobody published.

Derive the inventory from VPN routes, DNS, and app owners before you publish.

• Coexist before you cut over. Run ZTNA and the VPN side by side so a failed app reach falls back to

the VPN; never retire the VPN until the apps are proven.

• Pilot, gate, then expand in waves. Prove every entitled app on a small pilot, pass the gate, then widen entitlement in waves — assign once to a group, repeat everywhere.

• Keep a rollback ready. Access changes are disruptive; the fall-back is always "re-enable VPN and remove from the ZTNA `Access Group`" until the issue is fixed.

• Place publishers for performance and resilience. Put a `publisher` close to the apps in each segment and add a second per site before you cut over, so no single connector is a chokepoint.

• Report continuously. Run `NSZTNA-Report.ps1` on a schedule to audit who is entitled to what, verify posture, and catch any lingering VPN dependency after cutover.

• Defer console specifics to the vendor. Netskope screens and menus change; follow the named step and confirm the exact control in the official Netskope documentation and your tenant.

8. Deliverables Summary — Zero Trust Network Access (ZTNA) with Netskope Private Access

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	What ZTNA / Netskope Private Access is, the why vs the VPN, how it works, limitations
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing, plus the naming convention
Global Requirements (Network)	Category-wide prerequisites, published once
3. Implementation Guide	Ordered steps: publishers, private apps, least-privilege policy, pilot, validate, expand, cutover
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and safe remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`NSZTNA-Discovery.ps1`	Read-only: endpoint posture, current VPN routes/hosts, and candidate `publisher` hosts

Script	What it does
`NSZTNA-Validation.ps1`	Read-only: client state, entitlement-group membership, posture, and published-app reachability
`NSZTNA-Report.ps1`	Read-only: entitlement and usage report (who reaches which apps, posture, ZTNA usage)
`NSZTNA-Remediation.ps1`	Safe fixes (default `-WhatIf`): repair broken app access, e.g. add a user to an entitlement group
`NSZTNA-Set-AccessGroupTargeting.ps1`	Builds/updates the Entra `Access Group` entitlement groups and their membership

What the buyer receives

- This implementation guide as a branded Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable process: Assess to Configure to Pilot to Validate to Expand, with a pilot gate, a VPN coexistence-then-cutover migration, and an access-affecting rollback note.

Outcome

Users reach only the private apps they are entitled to — least-privilege, identity- and posture-based access with no network exposure — and the legacy VPN is retired, proven on a pilot before scaling.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

NSZTNA-Discovery.ps1

Purpose: Read-only assessment for a Netskope Private Access (ZTNA) VPN-replacement project. Captures the device's management/compliance posture signals, the routes/hosts the legacy VPN currently serves (a starting point for the private-app inventory), and whether the Netskope client is already present. Helps build the app inventory and confirm prerequisites before publishing apps. HOW TO RUN Elevated (for full network/route visibility): powershell -ExecutionPolicy Bypass -File .\NSZTNA-Discovery.ps1 EXPECTED RESULTS OS/build, domain-join/management state, active VPN/tunnel interfaces and their routes/hosts, Netskope client presence, and a note on candidate publisher placement. No changes are made. .NOTES Network — ZTNA / Netskope Private Access. Read-only. Authored the repeatable way. Netskope tenant-side discovery (publisher/app/policy state) is via the Netskope REST API and is deferred — see the flagged section below and the official Netskope documentation. #> [CmdletBinding()] param()

How to run: Elevated (for full network/route visibility): powershell -ExecutionPolicy Bypass -File .\NSZTNA-Discovery.ps1 EXPECTED RESULTS OS/build, domain-join/management state, active VPN/tunnel interfaces and their routes/hosts, Netskope client presence, and a note on candidate publisher placement. No changes are made. .NOTES Network — ZTNA / Netskope Private Access. Read-only. Authored the repeatable way. Netskope tenant-side discovery (publisher/app/policy state) is via the Netskope REST API and is deferred — see the flagged section below and the official Netskope documentation. #> [CmdletBinding()] param()

NSZTNA-Remediation.ps1

Purpose: Remediates the most common cause of a user not reaching an entitled private app: the user is missing from the Entra entitlement group ('Access Group - - ') that the Netskope access policy targets. Adds the named user to the named entitlement group via Microsoft Graph. SAFE BY DEFAULT: runs in -WhatIf mode and only previews the change; pass -Commit to apply. Access-affecting changes — confirm the rollback (re-enable VPN, remove from the group) before committing. HOW TO RUN Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\NSZTNA-Remediation.ps1 ^ -UserPrincipalName user@contoso.com -GroupName "Access Group - HR App - Pilot" Apply: ...same... -Commit Requires the Microsoft Graph PowerShell SDK and sign-in with GroupMember.ReadWrite.All. EXPECTED RESULTS In -WhatIf mode: a line showing the add that WOULD be made. With -Commit: the user is added to the entitlement group and the action is confirmed. Re-run NSZTNA-Validation.ps1 afterward. .NOTES Network — ZTNA / Netskope Private Access. Default -WhatIf. Authored the repeatable way. Netskope-side fixes (publisher health, app/policy config) are DEFERRED to the Netskope console. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$UserPrincipalName, [Parameter(Mandatory)][string]$GroupName, [switch]$Commit # omit for -WhatIf (preview-only) behavior)

How to run: Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\NSZTNA-Remediation.ps1 ^ -UserPrincipalName user@contoso.com -GroupName "Access Group - HR App - Pilot" Apply: ...same... -Commit Requires the Microsoft Graph PowerShell SDK and sign-in with GroupMember.ReadWrite.All. EXPECTED RESULTS In -WhatIf mode: a line showing the add that WOULD be made. With -Commit: the user is added to the entitlement group and the action is confirmed. Re-run NSZTNA-Validation.ps1 afterward. .NOTES Network — ZTNA / Netskope Private Access. Default -WhatIf. Authored the repeatable way. Netskope-side fixes (publisher health, app/policy config) are

DEFERRED to the Netskope console. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$UserPrincipalName, [Parameter(Mandatory)][string]$GroupName, [switch]$Commit # omit for -WhatIf (preview-only) behavior)

NSZTNA-Report.ps1

Purpose: Read-only reporting for a Netskope Private Access (ZTNA) deployment. Enumerates the Entra entitlement groups ('Access Group - - ') and their members from Microsoft Graph, so you can audit who is entitled to which private apps and confirm least-privilege. Optionally annotates members with device posture (compliant device count) from Graph. The ZTNA *usage* side (sessions/bytes per app) lives in Netskope and is DEFERRED to its REST API. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\NSZTNA-Report.ps1 ^ -GroupPrefix "Access Group" ^ -CsvPath .\ztna-access-report.csv Requires the Microsoft Graph PowerShell SDK and sign-in with Group.Read.All, User.Read.All, and DeviceManagementManagedDevices.Read.All (read-only). EXPECTED RESULTS A per-group listing of entitled members (and an optional CSV), plus a note that per-app usage metrics come from Netskope. No changes are made. .NOTES Network — ZTNA / Netskope Private Access. Read-only against Graph. Authored the repeatable way. #> [CmdletBinding()] param([string]$GroupPrefix = "Access Group", [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\NSZTNA-Report.ps1 ^ -GroupPrefix "Access Group" ^ -CsvPath .\ztna-access-report.csv Requires the Microsoft Graph PowerShell SDK and sign-in with Group.Read.All, User.Read.All, and DeviceManagementManagedDevices.Read.All (read-only). EXPECTED RESULTS A per-group listing of entitled members (and an optional CSV), plus a note that per-app usage metrics come from Netskope. No changes are made. .NOTES Network — ZTNA / Netskope Private Access. Read-only against Graph. Authored the repeatable way. #> [CmdletBinding()] param([string]$GroupPrefix = "Access Group", [string]$CsvPath)

NSZTNA-Set-AccessGroupTargeting.ps1

Purpose: Creates (if missing) and populates the Entra security groups that Netskope Private Access access policies target — the entitlement layer for least-privilege, per-app access. Group names follow the convention 'Access Group - - ' so policy targeting is consistent and auditable. SAFE BY DEFAULT: runs in -WhatIf mode and previews every create/add; pass -Commit to apply. Access-affecting — confirm the rollback before committing. HOW TO RUN Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\NSZTNA-Set-AccessGroupTargeting.ps1 ^ -GroupName "Access Group - HR App - Pilot" ^ -Members user1@contoso.com,user2@contoso.com Apply: ...same... -Commit Requires the Microsoft Graph PowerShell SDK and sign-in with Group.ReadWrite.All, GroupMember.ReadWrite.All, and User.Read.All. EXPECTED RESULTS In -WhatIf mode: the group create (if needed) and each member add that WOULD happen. With -Commit: the group exists and the listed members are added; non-listed members are left as-is unless -Exclusive is set. Re-run NSZTNA-Validation.ps1 / NSZTNA-Report.ps1 to confirm. .NOTES Network — ZTNA / Netskope Private Access. Default -WhatIf. Authored the repeatable way. Targeting this group from the Netskope access policy is done in the Netskope console (DEFERRED). #> [CmdletBinding()] param([Parameter(Mandatory)][string]$GroupName, [string[]]$Members = @(), [switch]$Exclusive, # if set, removes members not in -Members (still honors -WhatIf) [switch]$Commit # omit for -WhatIf (preview-only) behavior)

How to run: Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\NSZTNA-Set-AccessGroupTargeting.ps1 ^ -GroupName "Access Group - HR App - Pilot" ^ -Members user1@contoso.com,user2@contoso.com Apply: ...same... -Commit Requires the Microsoft Graph

PowerShell SDK and sign-in with Group.ReadWrite.All, GroupMember.ReadWrite.All, and User.Read.All. EXPECTED RESULTS In -WhatIf mode: the group create (if needed) and each member add that WOULD happen. With -Commit: the group exists and the listed members are added; non-listed members are left as-is unless -Exclusive is set. Re-run NSZTNA-Validation.ps1 / NSZTNA-Report.ps1 to confirm. .NOTES Network — ZTNA / Netskope Private Access. Default -WhatIf. Authored the repeatable way. Targeting this group from the Netskope access policy is done in the Netskope console (DEFERRED). #> [CmdletBinding()] param([Parameter(Mandatory)][string]$GroupName, [string[]]$Members = @(), [switch]$Exclusive, # if set, removes members not in -Members (still honors -WhatIf) [switch]$Commit # omit for -WhatIf (preview-only) behavior)

NSZTNA-Validation.ps1

Purpose: Read-only validation that a user/device is ready to reach private apps through Netskope Private Access. Confirms the Netskope client is present and steering, that the user is in the expected Entra entitlement group ('Access Group - - '), that the device reports a compliant/managed posture, and that a named published app resolves and is reachable. HOW TO RUN On a pilot device, signed in as (or specifying) the pilot user: powershell -ExecutionPolicy Bypass -File .\NSZTNA-Validation.ps1 ^ -UserPrincipalName user@contoso.com ^ -ExpectedGroup "Access Group - HR App - Pilot" ^ -AppFqdn hrapp.internal.contoso.com -AppPort 443 Entra checks require the Microsoft Graph PowerShell SDK and a sign-in with Group.Read.All and Device.Read.All (read-only scopes). EXPECTED RESULTS PASS/FAIL lines for: client present + steering, group membership, device posture, and the published app resolving/connecting. No changes are made. .NOTES Network — ZTNA / Netskope Private Access. Read-only. Authored the repeatable way. Netskope-side session/policy validation is via the Netskope console/REST API and is DEFERRED. #> [CmdletBinding()] param([string]$UserPrincipalName = $env:USERNAME, [string]$ExpectedGroup, [string]$AppFqdn, [int]$AppPort = 443)

How to run: On a pilot device, signed in as (or specifying) the pilot user: powershell -ExecutionPolicy Bypass -File .\NSZTNA-Validation.ps1 ^ -UserPrincipalName user@contoso.com ^ -ExpectedGroup "Access Group - HR App - Pilot" ^ -AppFqdn hrapp.internal.contoso.com -AppPort 443 Entra checks require the Microsoft Graph PowerShell SDK and a sign-in with Group.Read.All and Device.Read.All (read-only scopes). EXPECTED RESULTS PASS/FAIL lines for: client present + steering, group membership, device posture, and the published app resolving/connecting. No changes are made. .NOTES Network — ZTNA / Netskope Private Access. Read-only. Authored the repeatable way. Netskope-side session/policy validation is via the Netskope console/REST API and is DEFERRED. #> [CmdletBinding()] param([string]$UserPrincipalName = $env:USERNAME, [string]$ExpectedGroup, [string]$AppFqdn, [int]$AppPort = 443)